UNIVERSIDADE FEDERAL DO RIO GRANDE – FURG
CENTRO DE CIÊNCIAS COMPUTACIONAIS – C3
Curso de Engenharia de Computação

Otávio Chittolina de Almeida

FRONT-END DE COMPILADOR PASCAL-S COM PYTHON PLY

Relatório Técnico do Trabalho Prático 1

Relatório apresentado à disciplina de Compiladores, ministrada pelo Prof. Dr. Gerson Alberto Leiria Nunes, como parte da avaliação do semestre acadêmico 2026/2.

Rio Grande
2026

Resumo

Este relatório descreve o projeto e a implementação do front-end de um compilador para a linguagem Pascal-S. A solução foi desenvolvida em Python com a biblioteca PLY e integra análise léxica sob demanda, análise sintática LALR(1), tabela de símbolos com ambientes aninhados, verificação semântica de tipagem forte e recuperação de erros sintáticos em modo pânico. O analisador léxico lê o arquivo linha a linha, reconhece os padrões definidos para a linguagem e associa cada token identificador diretamente ao respectivo registro da tabela de símbolos. O parser implementa a gramática especificada por meio de produções recursivas à esquerda, enquanto as ações semânticas proíbem coerções implícitas entre `integer` e `real`. A recuperação usa o ponto e vírgula como símbolo de sincronização e permite relatar múltiplos erros em uma execução. A validação compreende dez programas de entrada com seus respectivos logs e três testes automatizados para recuperação e cálculo de deslocamentos.

Palavras-chave: compiladores; Pascal-S; PLY; análise LALR; tabela de símbolos; tipagem estática.

Sumário

1 Introdução **1**

2 Arquitetura da solução **1**

3 Analisador léxico **2**

3.1 Tokens e padrões 2

3.2 Tokenização sob demanda 3

3.3 Referência direta dos identificadores 3

4 Tabela de símbolos e ambientes **3**

4.1 Estruturas armazenadas 3

4.2 Declaração, duplicidade e omissão 4

4.3 Cálculo de deslocamentos 4

5 Analisador sintático LALR(1) **4**

5.1 Gramática reconhecida 4

5.2 Precedência e associatividade 5

5.3 Análise do conflito *dangling else* 6

6 Verificação semântica **6**

7 Tratamento e recuperação de erros **7**

7.1 Erros léxicos 7

7.2 Erros sintáticos e modo pânico 7
7.3 Erros semânticos 7

8 Validação experimental **8**
8.1 Ambiente e execução 8
8.2 Casos de teste e resultados 8

9 Decisões de projeto e limitações **9**

10 Conclusão **10**

A Rastreabilidade dos critérios de avaliação **10**

B Estrutura dos arquivos entregues **10**

1 Introdução

O trabalho consiste na construção do front-end completo de um compilador para Pascal-S, subconjunto imperativo e fortemente tipado de Pascal. Conforme a especificação da atividade [1], o produto deve reconhecer os tokens da linguagem, validar sua estrutura sintática, manter os atributos dos identificadores em uma tabela de símbolos e rejeitar programas semanticamente inválidos.

O escopo do projeto termina após a análise semântica. Portanto, o programa não gera código intermediário ou código de máquina e não executa o código-fonte de entrada. Quando todas as etapas terminam sem erro, o front-end informa que a compilação foi bem-sucedida e retorna código de saída zero. Erros léxicos, sintáticos ou semânticos produzem código de saída um.

Os objetivos técnicos atendidos são:

- tokenização preguiçosa, com leitura incremental do arquivo;
- reconhecimento da microssintaxe de Pascal-S, incluindo comentários;
- análise LALR(1) da gramática fornecida;
- armazenamento centralizado dos atributos dos identificadores;
- resolução de nomes em ambientes aninhados;
- rejeição de redeclarações, usos sem declaração e incompatibilidades de tipos;
- recuperação sintática pelo ponto e vírgula, com relato de múltiplos erros.

2 Arquitetura da solução

A implementação foi dividida em três módulos principais, cada um com uma responsabilidade direta:

`lex.py`
: Define tokens e expressões regulares, controla a leitura sob demanda, contabiliza linhas e colunas, descarta comentários e entrega ao parser os tokens reconhecidos.

`symbol_table.py`
: Define os registros de símbolos e os ambientes encadeados. Também realiza busca léxica, declaração, detecção de duplicidade e atribuição de deslocamentos.

`yacc.py`
: Define a gramática LALR(1), as ações semânticas, as regras de tipagem, o tratamento de erros e a interface de linha de comando.

O fluxo de dados começa em `compile_file`. Essa função cria uma tabela de símbolos vazia e um analisador léxico associado ao arquivo aberto. O parser solicita tokens conforme precisa de novos símbolos de entrada. Cada ocorrência de identificador já chega ao parser como referência ao registro correspondente na tabela. Durante as reduções, as ações da gramática completam ou consultam os atributos desses registros e propagam

tipos pelas expressões.

Essa organização mantém separadas a microssintaxe, a sintaxe e a gestão de atributos, ao mesmo tempo que permite a comunicação necessária entre as três etapas.

3 Analisador léxico

3.1 Tokens e padrões

O scanner foi implementado pela classe `PascalLexer`. As palavras reservadas são reconhecidas sem distinção entre maiúsculas e minúsculas por meio da conversão temporária do lexema para letras minúsculas. Identificadores comuns preservam sua grafia original.

Tabela 1: Categorias léxicas implementadas.

Categoria	**Tokens ou lexemas**	**Padrão adotado**
Palavras reservadas	`program`, `var`, `integer`, `real`, `begin`, `end`, `if`, `then`, `else`, `while`, `do`, `writeln`, `not`, `and`, `or`	Consulta em tabela hash após normalização para minúsculas.
Identificador	`ID`	`[a-zA-Z_][a-zA-Z0-9_]*`
Inteiro	`NUM_INT`	Um ou mais dígitos decimais.
Real	`NUM_REAL`	Dígitos, ponto e dígitos obrigatórios. A regra de real é aplicada antes da regra de inteiro.
Atribuição	`:=`	Token `ASSIGN`.
Relacionais	`=`, `<>`, `<`, `<=`, `>`, `>=`	Tokens `EQ`, `NE`, `LT`, `LE`, `GT` e `GE`.
Aritméticos	`+`, `-`, `*`, `/`	Tokens `PLUS`, `MINUS`, `TIMES` e `DIVIDE`.
Delimitadores	`:`, `;`, `,`, `.`, `(`, `)`	Tokens individuais para cada símbolo.

Espaços e tabulações são ignorados. Comentários iniciados por `//` são descartados até o fim da linha. Comentários entre chaves usam um estado léxico exclusivo denominado `comment`; nesse estado, o scanner descarta o conteúdo, continua contabilizando quebras de linha e só retorna ao estado normal ao encontrar `}`. Se o arquivo termina antes do fechamento, o scanner relata a posição inicial do comentário não terminado.

3.2 Tokenização sob demanda

O arquivo não é carregado integralmente nem convertido previamente em uma lista de tokens. O método `token()` tenta obter o próximo token do buffer corrente. Somente quando esse buffer termina é feita uma chamada a `readline()`, e a próxima linha passa a ser o novo buffer do PLY. Assim, a quantidade de entrada lida acompanha o avanço real do parser.

Como cada buffer contém uma linha física, `lexpos + 1` corresponde à coluna do token naquela linha. O contador `lineno` é incrementado nas regras de quebra de linha, inclusive dentro de comentários de bloco. Cada token entregue recebe também o atributo `column`, usado nas mensagens do parser.

Caracteres sem regra léxica são informados com linha, coluna e caractere. O scanner incrementa seu contador de erros, descarta um caractere e continua. A continuação é necessária para que o parser possa analisar o restante do arquivo.

3.3 Referência direta dos identificadores

Ao reconhecer um `ID`, o lexer chama `reference`. O valor do token deixa de ser apenas uma cadeia de caracteres e passa a ser o próprio objeto `Symbol`. Dessa forma, as ocorrências posteriores consultam ou alteram diretamente o registro central, satisfazendo a exigência de referências para entradas da tabela de símbolos.

4 Tabela de símbolos e ambientes

4.1 Estruturas armazenadas

Cada objeto `Symbol` contém os atributos apresentados na Tabela 2.

Tabela 2: Atributos de um símbolo.

Atributo	Finalidade
`name`	Grafia do identificador.
`scope`	Número do ambiente em que o símbolo foi criado.
`lineno`	Linha de declaração ou da primeira referência.
`type`	Tipo básico: `integer`, `real` ou ausente enquanto não declarado.
`offset`	Deslocamento relativo ao início do ambiente.
`kind`	Distingue variável e nome de programa.
`declared`	Indica se uma declaração válida já foi processada.

Um `Environment` contém um dicionário de símbolos, uma referência ao ambiente pai, um número de escopo e o próximo deslocamento livre. A tabela mantém um ambiente raiz e um ponteiro para o ambiente corrente. Ao reconhecer `begin`, o lexer cria um

ambiente filho; ao reconhecer `end`, retorna ao pai. A busca começa no ambiente corrente e segue os ponteiros `parent` até a raiz, implementando escopo estático.

A gramática atual permite declarações de variáveis apenas antes do bloco principal, como determina a especificação de Pascal-S. Mesmo assim, os ambientes aninhados deixam pronta a resolução de nomes caso uma extensão futura introduza declarações locais ou subprogramas.

4.2 Declaração, duplicidade e omissão

A primeira referência a um nome ainda desconhecido cria um registro incompleto no ambiente corrente. Quando a produção de declaração é reduzida, a operação `declare` preenche tipo, linha e deslocamento e marca o registro como declarado. Se o registro já estiver marcado, a declaração é duplicada e a compilação termina com erro semântico.

O uso de um registro incompleto em uma expressão ou no lado esquerdo de uma atribuição é detectado por `variable_type`. O mesmo ocorre se o símbolo representar o nome do programa em vez de uma variável. Nesses casos, é emitida uma mensagem de variável não declarada.

4.3 Cálculo de deslocamentos

Cada ambiente começa com deslocamento zero. Uma variável `integer` reserva 4 bytes e uma variável `real` reserva 8 bytes. O deslocamento é atribuído antes de avançar o contador, sem preenchimento de alinhamento. Por exemplo, a sequência `integer`, `integer`, `real`, `real` recebe os deslocamentos 0, 4, 8 e 16.

Os contadores são independentes por ambiente e uma tentativa de declaração duplicada não consome espaço. Como o PLY pode solicitar antecipadamente o token `begin`, o ambiente corrente pode já ser o filho do bloco quando a última declaração é reduzida. Por isso, a operação de declaração percorre os pais até encontrar o ambiente registrado no atributo `scope` do símbolo. Essa decisão mantém o deslocamento no escopo correto apesar do lookahead do parser.

5 Analisador sintático LALR(1)

5.1 Gramática reconhecida

A gramática abaixo reproduz a EBNF da especificação. Na implementação do PLY, as repetições foram convertidas em produções recursivas à esquerda, reduzindo o crescimento da pilha do analisador.

Listing 1: Gramática EBNF de Pascal-S.

```
program ::= "program" identifier ";" decl_list block "."
decl_list ::= [ "var" decl_group_list ]
decl_group_list ::= decl_group { decl_group }
decl_group ::= id_list ":" type ";"
id_list ::= identifier { "," identifier }
type ::= "integer" | "real"
block ::= "begin" stmt_list "end"
stmt_list ::= stmt { ";" stmt }
stmt ::= assign_stmt | if_stmt | while_stmt |
                  write_stmt | block | empty
empty ::=
assign_stmt ::= identifier ":=" expr
if_stmt ::= "if" expr "then" stmt [ "else" stmt ]
while_stmt ::= "while" expr "do" stmt
write_stmt ::= "writeln" "(" expr_list ")"
expr_list ::= expr { "," expr }
expr ::= simple_expr [ relop simple_expr ]
simple_expr ::= term { addop term }
term ::= factor { mulop factor }
factor ::= identifier | number | "(" expr ")" |
                  "not" factor
relop ::= "=" | "<>" | "<" | "<=" | ">" | ">="
addop ::= "+" | "-" | "or"
mulop ::= "*" | "/" | "and"
```

As produções adicionais que contêm o símbolo especial `error` não ampliam a linguagem aceita: elas existem apenas para recuperar a análise depois de uma construção inválida. A presença de qualquer erro mantém o resultado global da compilação como falha.

5.2 Precedência e associatividade

A própria estrutura da gramática determina a precedência. Fatores são reduzidos antes de termos; termos, antes de expressões simples; e expressões simples, antes de relações. Consequentemente, `not` possui o nível mais alto, `*`, `/` e `and` vêm em seguida, depois aparecem `+`, `-` e `or`, e os relacionais ocupam o nível mais baixo. As regras recursivas à esquerda de `term` e `simple_expr` tornam os operadores desses níveis associativos à esquerda. A produção de `expr` permite no máximo um operador relacional.

Essa estratificação dispensa declarações artificiais de precedência para os operadores aritméticos e lógicos: formas de derivação concorrentes não são geradas.

5.3 Análise do conflito *dangling else*

A construção condicional contém a ambiguidade clássica conhecida como *dangling else*. Após reconhecer `if expr then stmt`, com `else` como próximo token, um parser poderia reduzir imediatamente o comando condicional sem `else` ou deslocar o token para formar a alternativa completa.

A implementação atribui à alternativa incompleta a precedência fictícia `IFX`, por meio de `%prec IFX`. A tabela de precedência declara `IFX` antes de `ELSE`; portanto, `ELSE` possui precedência maior. Diante desse token, o PLY escolhe o deslocamento. O `else` é, assim, associado ao `if` não encerrado mais próximo, que é o comportamento convencional de Pascal.

Durante a geração de diagnóstico, o PLY mostrou a alternativa rejeitada:

```
! ELSE [ reduce using rule 26 (if_stmt -> IF expr THEN stmt .) ]
```

Não foram emitidos avisos de conflito *shift/reduce* ou *reduce/reduce*. A linha acima registra a ação de redução descartada pela precedência explícita; a ação efetiva é o deslocamento. Logo, a tabela LALR(1) final não contém conflitos não resolvidos.

6 Verificação semântica

As ações semânticas são executadas durante as reduções. Cada não terminal de expressão sintetiza um tipo. Constantes inteiras produzem `integer`, constantes reais produzem `real`, e identificadores produzem o tipo armazenado em seu registro.

As regras aplicadas são:

1. Operadores `+`, `-`, `*` e `/` exigem dois operandos numéricos do mesmo tipo e conservam esse tipo no resultado.
2. Operadores relacionais exigem dois operandos numéricos do mesmo tipo e produzem o tipo interno `boolean`.
3. `and` e `or` exigem dois operandos booleanos; `not` exige um operando booleano.
4. As condições de `if` e `while` devem ser booleanas.
5. Uma atribuição só é aceita quando o tipo da variável é exatamente igual ao tipo da expressão.

Não existe coerção implícita em nenhuma direção. Assim, tanto atribuir um `real` a um `integer` quanto atribuir um `integer` a um `real` é erro. A mesma regra de igualdade vale para os operandos de operações aritméticas e relacionais.

Ao detectar um erro semântico, a função `semantic_error` informa linha, coluna e motivo e lança `CompileError`. A captura em `compile_file` encerra coordenadamente a compilação com resultado de falha, atendendo à exigência de aborto imediato para incompatibilidades de tipo.

7 Tratamento e recuperação de erros

7.1 Erros léxicos

Um caractere ilegal gera mensagem com sua linha e coluna e é descartado. O scanner continua até o fim da entrada. Como o descarte pode deixar uma sequência sintaticamente inválida, um mesmo programa também pode produzir erro sintático. No caso `10 @ 5`, por exemplo, o `@` é descartado e o segundo número se torna inesperado na expressão.

Comentários de bloco sem chave de fechamento são detectados quando `readline()` alcança o fim do arquivo. A mensagem aponta para a chave que iniciou o comentário.

7.2 Erros sintáticos e modo pânico

O callback `p_error` incrementa o contador de erros e informa o token inesperado, seu tipo, linha e coluna. Se o problema ocorre no fim do arquivo, a posição é obtida do último buffer do scanner.

O ponto e vírgula é o símbolo estável de sincronização. As produções de recuperação cobrem os dois contextos relevantes:

```
decl_group : error SEMICOLON

stmt_list : error SEMICOLON stmt
          | stmt_list SEMICOLON error SEMICOLON stmt
```

Ao ocorrer uma falha, o mecanismo do PLY remove estados ou descarta tokens até que uma dessas produções possa consumir `SEMICOLON`. Na declaração, uma chamada a `errok()` libera imediatamente o parser para relatar o próximo erro. Nas listas de comandos, as produções permitem retomar a partir do comando posterior ao delimitador.

Essa estratégia preserva o ambiente léxico corrente e permite encontrar erros em blocos aninhados. O caso de teste 10 demonstra que duas declarações inválidas não impedem o relato de três erros posteriores em comandos.

7.3 Erros semânticos

Diferentemente dos erros sintáticos, os erros semânticos são impeditivos e encerram a análise na primeira ocorrência, como solicitado na especificação. Essa política evita propagar tipos inválidos e produzir diagnósticos em cascata sem utilidade.

8 Validação experimental

8.1 Ambiente e execução

A implementação utiliza Python 3 e PLY 3.11, fixado em `requirements.txt`. Um ambiente pode ser preparado e o compilador pode ser executado com os comandos seguintes:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python yacc.py tests/01-success.pas
```

O conjunto automatizado é executado por:

```
.venv/bin/python -m unittest discover -s tests -v
```

8.2 Casos de teste e resultados

Os dez arquivos de entrada possuem logs correspondentes em `tests/output`. A Tabela 3 resume o comportamento observado.

Tabela 3: Bateria de testes do front-end.

Caso	Objetivo	Resultado registrado
01	Programa completo com variáveis inteiras e reais, laço e bloco aninhado.	Compilação bem-sucedida; código 0.
02	Caractere ilegal `@`.	Erro léxico em 5:16 e erro sintático decorrente em 5:18; código 1.
03	Expressão real atribuída a variável inteira.	Erro semântico em 8:5; código 1.
04	Dois comandos sintaticamente inválidos.	Dois erros sintáticos, em 6:10 e 7:12, na mesma execução; código 1.
05	Maiúsculas e minúsculas em palavras reservadas, comentários, operadores lógicos, blocos e *dangling else*.	Compilação bem-sucedida; código 0.
06	Dupla declaração de variável no mesmo escopo.	Erro semântico em 4:5; código 1.
07	Uso de variável não declarada.	Erro semântico em 3:5; código 1.
08	Variável inteira atribuída a variável real.	Erro semântico em 7:5; código 1.

Caso	Objetivo	Resultado registrado
09	Comentário de bloco sem fechamento.	Erro léxico apontando o início em 4:6; código 1.
10	Duas declarações e três comandos inválidos, inclusive em bloco aninhado.	Cinco erros sintáticos relatados na mesma execução; código 1.

Os três cenários mínimos solicitados pela especificação correspondem aos casos 01, 02 e 03: sucesso completo, erro léxico por caractere ilegal e erro semântico por incompatibilidade de tipos. Os demais casos ampliam a cobertura para recuperação sintática, comentários, escopos, operadores lógicos, redeclaração, variável não declarada e proibição de coerção nas duas direções.

Além dos programas e logs, três testes unitários verificam propriedades internas:

- os deslocamentos 0, 4, 8 e 16 no programa de sucesso;
- o reinício do deslocamento em ambiente filho e a ausência de alocação após declaração duplicada;
- a recuperação após declarações inválidas, as cinco posições esperadas e o retorno correto ao ambiente raiz.

Na execução de validação, os três testes terminaram com estado `OK`. Também foram reproduzidos os dez logs de entrada e saída e a geração das tabelas LALR não emitiu conflitos não resolvidos.

9 Decisões de projeto e limitações

As principais decisões foram tomadas para manter correspondência direta com a especificação e reduzir estados ou estruturas desnecessárias:

- a precedência de operadores é representada pela hierarquia da gramática; apenas o *dangling else* usa precedência declarativa;
- os identificadores carregam referências Python para seus registros, sem cópias intermediárias dos atributos;
- a leitura linha a linha fornece comportamento sob demanda e torna `lexpos` diretamente utilizável como coluna;
- os deslocamentos são relativos ao ambiente e não aplicam alinhamento, pois não há geração de código ou modelo de máquina no escopo deste trabalho;
- ambientes aninhados já são criados para cada bloco, embora a gramática atual só admita declarações antes do bloco principal;
- erros sintáticos são acumulados; erros semânticos encerram a análise imediatamente, seguindo as políticas distintas da especificação.

O front-end não constrói uma árvore sintática abstrata persistente. As verificações necessárias são realizadas por tradução dirigida pela sintaxe, e os tipos são propagados durante as reduções. Para os requisitos do trabalho, essa abordagem evita armazenar uma estrutura que não seria usada por uma etapa posterior.

10 Conclusão

O front-end implementado cobre a microssintaxe e a gramática definidas para Pascal-S e integra as análises léxica, sintática e semântica por meio do PLY. A leitura incremental evita a criação de uma fita estática de tokens. A tabela de símbolos centraliza nome, escopo, tipo, linha e deslocamento, enquanto as ocorrências de identificadores mantêm referências diretas a esses registros.

A gramática recursiva à esquerda representa a precedência dos operadores, e a precedência explícita entre `IFX` e `ELSE` resolve o único ponto clássico de ambiguidade. A tipagem forte rejeita todas as misturas entre `integer` e `real`. Por fim, a sincronização em ponto e vírgula permite relatar vários erros sintáticos sem comprometer a coerência dos escopos.

Os dez programas com logs e os três testes unitários exercitam tanto os cenários obrigatórios quanto casos adicionais de borda. Os resultados obtidos confirmam o atendimento aos requisitos funcionais e às decisões de engenharia descritas neste relatório.

A Rastreabilidade dos critérios de avaliação

Critério	**Peso**	**Evidência no projeto e no relatório**
Analisador léxico	25%	Seção 3; padrões, comentários, posições e leitura sob demanda em `lex.py`.
Tabela de símbolos e tipos	20%	Seção 4; referências diretas, ambientes, atributos, deslocamentos, duplicidade e omissão em `symbol_table.py`.
Analisador sintático	35%	Seção 5; gramática completa, recursão à esquerda, precedência e análise de conflitos em `yacc.py`.
Erros e sincronização	10%	Seção 7; diagnóstico por posição e recuperação em ponto e vírgula.
Relatório e organização	10%	Arquitetura, decisões, conflitos, validação e rastreabilidade documentados neste texto.

B Estrutura dos arquivos entregues

```
lex.py
yacc.py
```

```
symbol_table.py
requirements.txt
README.md
tests/
  01-success.pas
  02-lexical-error.pas
  03-type-error.pas
  04-syntax-recovery.pas
  05-lexer-scopes-logic.pas
  06-duplicate-declaration.pas
  07-undeclared-variable.pas
  08-integer-to-real.pas
  09-unterminated-comment.pas
  10-declaration-recovery.pas
  test_frontend.py
  output/
    01-success.log ... 10-declaration-recovery.log
```

Referências

[1] NUNES, Gerson Alberto Leiria. **Trabalho Prático 1 – Front-End Completo com PLY: especificação técnica e diretrizes**. Universidade Federal do Rio Grande, Centro de Ciências Computacionais, 2026.

[2] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compilers: Principles, Techniques, and Tools**. 2. ed. Boston: Addison-Wesley, 2006.

[3] BEAZLEY, David M. **PLY (Python Lex-Yacc): Documentation**. Versão 3.11. Disponível em: `https://www.dabeaz.com/ply/ply.html`. Acesso em: 24 out. 2026.